

Lab5 Reflection

Task 1

Time comparison

Task 1: iterative VS recursive fibonacci numbers

iterative 40th fibonacci number & time cost

102334155

0.000000 seconds

Recursive 40th fibonacci number & time cost

102334155

0.451623 seconds

Explanation

The recursive version of the function is so much slower because its cost grows exponentially, requiring two calls of itself for every iteration. The iterative version avoids this by storing the previous value and adding it to the current.

Task 2

Result of n = 200000

Task 2: recursive factorial TCO vs no TCO

Recursive factorial with TCO time cost

0

0.002163 seconds

Recursive factorial time cost

Warning: detected a stack overflow; program state may be corrupted, so further execution might be unreliable.

ERROR: LoadError: StackOverflowError:

Stacktrace:

[1] fact(n::Int64) (repeats 1025 times)

@ Main ~/school/spring2026/CS431/week5/funtional_logic.jl:51

Explanation

The standard version triggers a stack overflow somewhere between 200000 and 300000 iterations. The TCO version of the function survives past 500000 iterations because the compiler is able to use the same stack frame.

Note while the TCO version survives a stack overflow it returns 0 because we quickly outgrow the value storable by an Int64

Task 3

Compare the "elegance" of this code vs. a while loop approach.

I think the recursive version of this function feels more elegant because it avoids having to deal with any extra variables for managing the current and next node. Instead allowing you to just pass the next node as the argument to the recursive function.

Reflection

Your reflection on when recursion becomes a "liability" in production environments.

I think recursion becomes a liability in production environments when it is used in situation where the number of iterations needed to complete a task is unknown and tail call optimization is not available. In my opinion I think recursion should be used with caution and reserved for cases with known input sizes when provide more understandable code.